Rhilo Sotto
2/19/2025
COMPE 470L

Lab 4 Report

Introduction

In this lab we design a UART Transmitter with a Clock Scaler to transmit our last name at 9600 bit/s. We implement it in Verilog alongside a testbench to test our Verilog module.The module takes in 8 parallel data bits and transmits them as an 8-bit sequence, alongside a start and a stop bit to indicate when transmission begins and ends.

Verilog Code

UART Transmitter Module

```
module UART_Transmitter #(parameter CLKS_PER_BIT = 10) (
    input CLK,
    input data_valid,
    input [7:0] data_in,
    output reg active,
    output reg data_out,
    output [2:0] cur_state, output [2:0] counter // state and counter
    );

    localparam Idle = 3'b000, Start = 3'b001, Data = 3'b010, End = 3'b011;
    reg [2:0] state = Idle;
    reg [2:0] data_counter = 0;
    reg [7:0] internal_data;

     // view state and counter as output
     assign cur_state = state;
     assign counter = data_counter;

     reg [7:0] BAUD_RATE;
     reg [7:0] Clock_Count;
     reg SEND_BIT;


     always @(posedge CLK) begin // CLOCK SCALING
        if (Clock_Count < CLKS_PER_BIT-1) begin
```

```
      Clock_Count <= Clock_Count + 1;
      SEND_BIT <= 1;
    end
    else begin
      Clock_Count <= 0;
      SEND_BIT <= 0;
    end
end

always @(posedge SEND_BIT) begin // next state logic
    case(state)

      Idle: begin
         state <= (data_valid) ? Start : Idle;
      end

      Start: begin
         state <= Data;
         internal_data <= data_in;
         data_counter <= 0;
      end

      Data: begin
         state <= (data_counter == 7) ? End : Data;
         data_counter <= (data_counter + 1) % 8;
      end

      End: begin
         state <= (data_valid) ? Start : Idle;
      end

      default: state <= Idle;

    endcase
end

always @(*) begin // output logic
   case(state)
      Idle: begin
         data_out <= 1'b1; // idle high
         active <= 1'b0;
      end
      Start: begin
         data_out <= 1'b0; // drops low
```

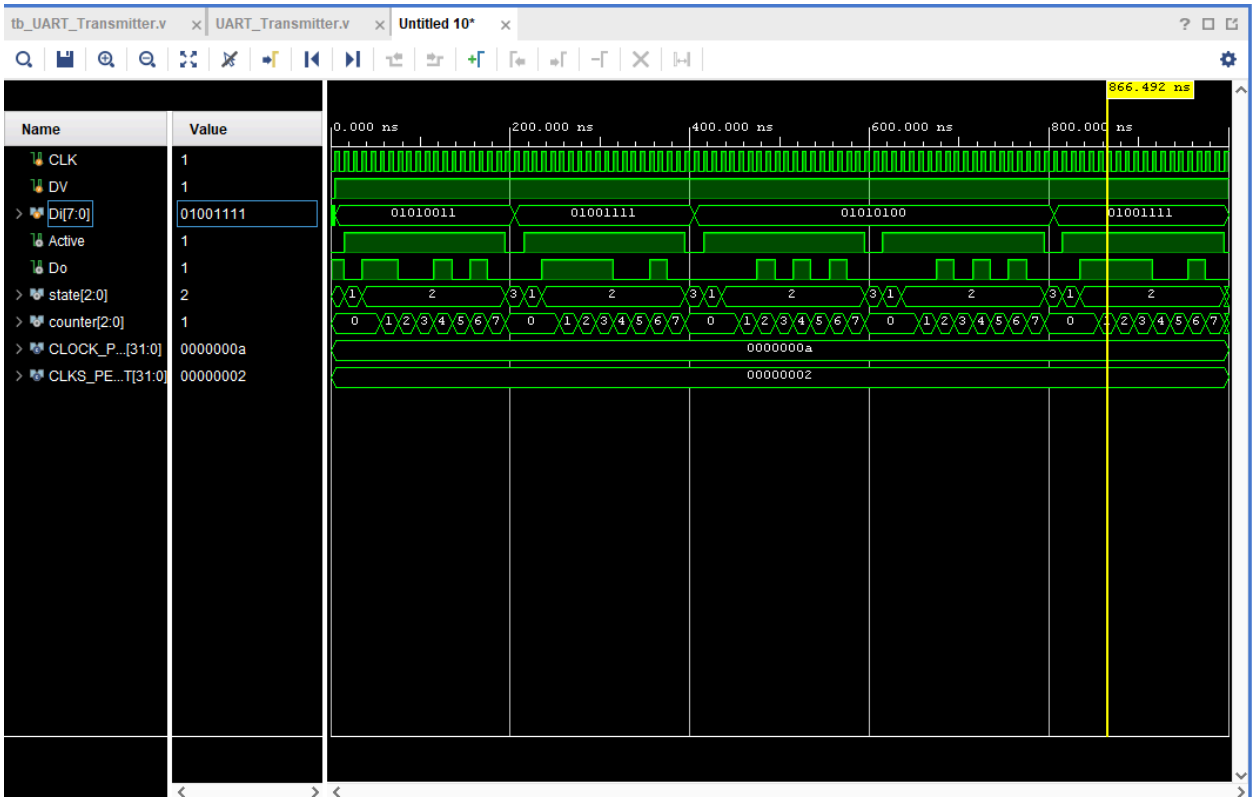
```
            active <= 1'b1;
        end
        Data: begin
            data_out <= internal_data[data_counter];
        end

        End: begin
            active <= 1'b0;
        end
      endcase
    end



endmodule
```

Testbench

```
`timescale 1ns / 1ns

module tb_UART_Transmitter;


localparam CLOCK_PERIOD_NS = 10, CLKS_PER_BIT = 2;

reg CLK = 0;
always #(CLOCK_PERIOD_NS/2) CLK <= ~CLK;

reg DV = 0;
reg [7:0] Di = 8'b00000000;
wire Active, Do;
wire [2:0] state, counter;

UART_Transmitter #(.CLKS_PER_BIT(CLKS_PER_BIT)) Tx(.CLK(CLK), .data_valid(DV),
.data_in(Di), .active(Active), .data_out(Do),
.cur_state(state), .counter(counter));

initial begin
    /*
    S = 8'b01010011
    O = 8'b01001111
    T = 8'b01010100
    T = 8'b01010100
    O = 8'b01001111
    */

    #5;
    DV = 1;
    Di = 8'b01010011; // S
    #200;
    Di = 8'b01001111; // O
    #200;
    Di = 8'b01010100; // T
    #200;
    Di = 8'b01010100; // T
    #200;
    Di = 8'b01001111; // O
    #200;
    DV = 0;
    #100;
```

```
    $finish;
end

endmodule
```

Results



Conclusion

In this lab, we designed a UART Transmitter with a Clock Scaler to transmit our last name at 9600 bit/s in Verilog and successfully calculated the required clock cycles per second given a 100MHz clock frequency such as that on the Basys3. I successfully showcased my simulation demo of my last name as transmission bytes in uppercase ASCII, which was transmitted in reverse order serially.